Object-Oriented Programming — Foundations

BS./BSc. in Applied AI and Data Science — Week 1 (Dr. Debasis Das, IIT Jodhpur)

Formatted by: **Heisenberg** 😎

Index

- Course Overview — Who's teaching this, and why you should care.
- Syllabus, Lab, and Evaluation — The fine print: what's on the exam and how much it's worth.
- Introduction to Computers — CPUs, bytes, and other things your laptop does while you're not looking.
- Programming Languages — From 0s and 1s to something an actual human can read.
- Translators: Assemblers, Interpreters, and Compilers — How your code learns to speak binary.
- Operating Systems — The referee standing between you and your hardware.
- Introduction to Object-Oriented Programming — Where everything becomes an object (yes, even the car).
- The Four Pillars of OOP — Encapsulation, Inheritance, Polymorphism, Abstraction — OOP's own Avengers team.
- Advantages of OOP — Why programmers bother with all of this.
- Features of Java — What makes Java, well, Java.

Course Overview

This course is taught by **Dr. Debasis Das**, Associate Professor in the Department of Computer Science and Engineering at IIT Jodhpur. The subject is **Object-Oriented Programming (OOP)** — an advanced-level programming paradigm — implemented here using the **Java programming language**, which is described as a *fully* object-oriented language.

Course Objectives

- Introduce the principles of object-oriented programming using Java.
- Teach object-oriented **design, data abstraction, inheritance, polymorphism, exception handling, generic programming, collections, multithreading, graphical user interface (GUI) development, database connectivity**, and modern Java programming techniques.

Syllabus, Lab, and Evaluation

This course covers fundamental Object-Oriented Programming concepts including class hierarchy, inheritance, overriding, polymorphism, and abstract classes. It delves into collections, iterators, generics, and callbacks. Further topics include cloning, I/O serialization, and package management, alongside exception handling and concurrent programming. The laboratory component focuses on implementing data structures in Java, empirically verifying and comparing the runtime performance and asymptotic behavior of various data structures and algorithms using real-life applications.

Evaluation Scheme

Component	Weightage (%)	Remarks
Assignments	20%	Two or more assignments, conducted online
Quizzes	20%	Two or more quizzes, conducted online
Major Exam	60%	Closed book, written examination, physical mode; scientific calculators permitted

Introduction to Computers

The central theme of this course is learning **how to solve problems by writing programs**. Before diving into Java and OOP specifically, it helps to have the underlying vocabulary of computers and software straight.

What Is Programming?

Programming means to create (or develop) software, which is also called a **program**. In basic terms, software contains the instructions that tell a computer — or any computerized device — what to do. In other words, the program is responsible for *what* the computer does; the computer itself only stores and executes those instructions.

What Is a Computer?

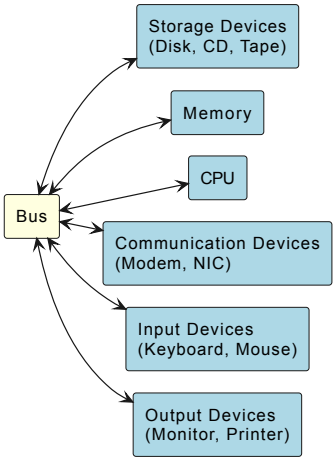
A **computer** is an electronic device that stores and processes data. A computer is made up of both:

- **Hardware** — the visible, physical elements of the computer.
- **Software** — the invisible instructions that control the hardware and make it perform specific tasks.

Knowing computer hardware isn't strictly essential to learning a programming language, but it does help you understand the effects a program's instructions have on the computer and its components.

Major Hardware Components of a Computer

A computer consists of a **CPU**, **memory**, **storage devices**, **input devices**, **output devices**, and **communication devices** — all connected via a common **bus**, which lets every component exchange information and communicate with the others.



Central Processing Unit (CPU)

The **CPU** is the computer's brain. It retrieves instructions from memory and executes them. It usually has two components:

- **Control Unit** — controls and coordinates the actions of the other components.
- **Arithmetic/Logic Unit (ALU)** — performs numeric operations (addition, subtraction, multiplication, division) and logical operations (comparisons).

Bits and Bytes

A computer's storage capacity is measured in **bytes** and multiples of the byte:

Unit	Approximate Size
Kilobyte (KB)	~1,000 bytes
Megabyte (MB)	~1 million bytes
Gigabyte (GB)	~1 billion bytes
Terabyte (TB)	~1 trillion bytes
Petabyte (PB)	1,000 Terabytes
Exabyte (EB)	1,000 Petabytes
Zettabyte (ZB)	1,000 Exabytes
Yottabyte (YB)	1,000 Zettabytes

> **Note for Review:** Each unit is roughly a thousand times the previous one (informally; technically some contexts use 1024). You don't need to memorize the whole ladder — just know the order: KB → MB → GB → TB → PB → EB → ZB → YB.

Memory

A computer's **memory** consists of an ordered sequence of bytes used to store both the program itself and the data the program is working with. Think of memory as the computer's *work area* for executing a program — a program and its data must be moved into memory before the CPU can execute them.

Memory Address	Memory Content (binary)	Meaning
2000	01000011	Encoding for character `C`
2001	01110010	Encoding for character `r`
2002	01100101	Encoding for character `e`
2003	01110111	Encoding for character `w`
2004	00000011	Encoding for number `3`

Storage Devices

There are three main types of storage devices:

- **Magnetic disk drives**
- **Optical disc drives** (CD and DVD)
- **USB flash drives**

Programming Languages

A computer program (software) is a set of instructions that tells a computer what to do. Since computers don't understand human languages, programs must be written in a language the computer can use. Hundreds of programming languages exist, all designed to make the programming process easier for people — but every one of them must eventually be converted into instructions the computer can actually execute.

There are three broad categories of programming language:

1. **Machine Language**
2. **Assembly Language**
3. **High-Level Language**

Machine Language

A computer's **native language**, which differs across computer types, is its *machine language* — a set of built-in primitive instructions written entirely in **binary code**. To give a computer an instruction in its native language, you must enter that instruction as binary code, for example:

```
1101101010011010
```

Assembly Language

Programming directly in machine language is tedious, and machine-language programs are very difficult to read and modify. **Assembly language** was created in the early days of computing as a more manageable alternative. It uses a short descriptive word, called a **mnemonic**, to represent each machine-language instruction — for instance, `add` typically means "add numbers" and `sub` means "subtract numbers." To add the numbers 2 and 3, you might write:

```
add 2, 3, result
```

High-Level Language

In the 1950s, a new generation of programming languages called **high-level languages** emerged. They are **platform independent** — a program written in a high-level language can run on different types of machines. High-level languages are English-like, and easy to learn and use; their instructions are called **statements**. For example, a statement that computes the area of a circle with radius 5:

```
area = 5 * 5 * 3.14159;
```

Popular High-Level Programming Languages

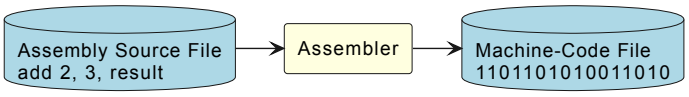
Language	Description
Ada	Named for Ada Lovelace; developed for the Department of Defense, used mainly in defense projects.
BASIC	Beginner's All-purpose Symbolic Instruction Code — designed to be learned and used easily by beginners.
C	Developed at Bell Laboratories; combines the power of assembly language with the ease of use and portability of a high-level language.
C++	An object-oriented language, based on C.
C#	Pronounced "C Sharp"; a hybrid of Java and C++, developed by Microsoft.
COBOL	COmmon Business Oriented Language; used for business applications.
FORTRAN	FORmula TRANslation; popular for scientific and mathematical applications.
Java	Developed by Sun Microsystems (now Oracle); widely used for developing platform-independent internet applications.
Pascal	Named for Blaise Pascal; a simple, structured, general-purpose language primarily for teaching programming.
Python	A simple, general-purpose scripting language, good for writing short programs.
Visual Basic	Developed by Microsoft; enables programmers to rapidly develop graphical user interfaces.

Translators: Assemblers, Interpreters, and Compilers

Whatever language a program is written in, it eventually has to become machine code. Different language levels use different translators to get there.

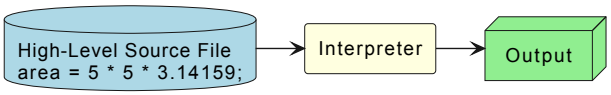
Assembler

An **assembler** translates assembly-language instructions into machine code.



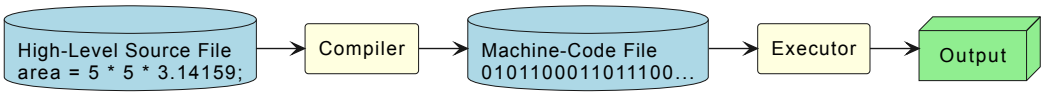
Interpreter

An **interpreter** translates and executes a program **one statement at a time**.



Compiler

A **compiler** translates the *entire* source program into a machine-language file first; that file is then run by an executor to produce output.

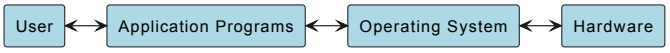


> **Note for Review:** The key distinction that often shows up in exams — an **interpreter** translates and runs code statement-by-statement (no separate machine-code file is produced), while a **compiler** translates the whole program up front into a machine-code file, which is then executed separately. Java, notably, uses a mix of both (compiling to bytecode, then the JVM interprets/JIT-compiles it) — a detail worth keeping in mind as the course goes deeper into Java's execution model.

Operating Systems

The **operating system (OS)** is the most important program that runs on a computer — it manages and controls a computer's activities and resources. The most popular operating systems for general-purpose computers are **Microsoft Windows**, **Mac OS**, and **Linux**.

Application programs, such as a web browser or a word processor, cannot run unless an operating system is installed and running on the computer. Users and application programs access the computer's hardware **via** the operating system:



Introduction to Object-Oriented Programming

With the hardware/software fundamentals out of the way, the course turns to its actual subject: **Object-Oriented Programming (OOP)**, implemented in Java.

Topics to be covered under Basic OOP (Java):

- Introduction to Object-Oriented Programming
- Features of Java
- Java Virtual Machine (JVM), JDK, and JRE
- Structure of a Java program
- Data types, variables, operators, and expressions
- Input/Output in Java
- Control statements (`if` , `switch` , loops)

What Is a Programming Paradigm?

A **programming paradigm** is a style or approach to programming. Examples include:

Paradigm	Example Languages
Procedural Programming	C
Object-Oriented Programming	Java, C++
Functional Programming	Scala, Haskell
Logic Programming	Prolog

What Is Object-Oriented Programming?

Object-Oriented Programming organizes software as a collection of **objects**, each of which contains both **data** and **methods**. An object consists of three components:

- **State** — its attributes/data
- **Behavior** — its methods/functions
- **Identity** — a unique reference

Real-Life Example: A Car

Take a **Car** as an object:

- **State:** Color, Model, Speed
- **Behavior:** `Start()` , `Accelerate()` , `Brake()`

In Java, this maps directly onto a class:

```
class Car {
    String color;
    int speed;

    void start() {
        System.out.println("Car Started");
    }

    void brake() {
        System.out.println("Car Stopped");
    }
}
```

Here, `color` and `speed` represent the car's **state** (its variables), while `start()` and `brake()` represent its **behavior** (its methods). This is the fundamental shape of every Java program built on OOP principles: a **class** defines the blueprint, from which **objects** are created.

The Four Pillars of OOP

Every object-oriented language — Java included — rests on four foundational pillars:

1. **Encapsulation**
2. **Inheritance**
3. **Polymorphism**
4. **Abstraction**

1. Encapsulation

Encapsulation means binding data and methods together, and typically restricting direct access to the data (making it `private`) while exposing controlled access through methods.

```
class Student {
    private int age;

    public void setAge(int age) {
        this.age = age;
    }

    public int getAge() {
        return age;
    }
}
```

Advantages: data hiding, better security, easy maintenance.

2. Inheritance

Inheritance allows one class to inherit the properties of another.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
```

Here, `Dog extends Animal` means `Dog` automatically inherits everything `Animal` has — so a `Dog` object can call both `eat()` (inherited from `Animal`) and `bark()` (its own method). Executing both would produce:

```
Eating
Barking
```

> **Note for Review:** "Extends" is the keyword to watch for — it signals that the class on the left (the *child*/*derived* class) inherits from the class on the right (the *parent*/*base* class). The child gets everything the parent has, plus whatever new members it defines itself.

3. Polymorphism

Polymorphism literally means "one interface, many implementations" — the same method name behaves differently depending on how it's called.

```
class MathDemo {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Both methods are named `add`, but one works on integers and the other on doubles — the correct version is selected automatically based on the arguments passed in. This particular flavor of polymorphism (same name, different parameter types) is called **method overloading**.

4. Abstraction

Abstraction means showing essential information while hiding implementation details.

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

`Shape` declares *that* every shape can be drawn (`draw()`), without saying *how*. `Circle` — a concrete subclass — supplies the actual implementation, producing:

```
Drawing Circle
```

> **Note for Review:** Abstraction and Encapsulation are often confused. Encapsulation is about **hiding data** (bundling state with the methods that control it). Abstraction is about **hiding complexity** (exposing *what* something does, not *how* it does it). An abstract class is one concrete tool for achieving abstraction.

Advantages of OOP

Object-oriented programming offers several practical benefits:

- **Reusability** — via inheritance, existing functionality can be reused rather than rewritten.
- **Security** — encapsulation keeps internal data protected from outside interference.
- **Modularity** — code is organized into self-contained classes/objects.
- **Flexibility** — polymorphism allows the same interface to adapt to different situations.
- **Easy Maintenance** — changes tend to stay localized within a class.
- **Scalability** — new classes and features can be added without disrupting existing code.

Features of Java

Java was developed by **Sun Microsystems** (now part of **Oracle**) in **1995**. Its major features include:

- **Simple** — easy syntax; automatic memory management (garbage collection).
- **Object-Oriented** — everything revolves around classes and objects.
- **Platform Independent** — "compile once, run anywhere," since Java compiles to bytecode that any JVM can execute regardless of the underlying machine.

End of Lecture 1. Next class: a deeper dive into object-oriented programming fundamentals — classes, objects, and the mechanics behind inheritance, overloading, and overriding.